

MID-TERM PROJECT SPECIFICATION

502094 - Software Deployment, Operations and Maintenance

Cloud Migration & Containerization

Don't panic if this looks long. It's detailed so you have all the necessary information—please read it carefully.

Last updated on Dec 15, 2025

1. Introduction and Purpose

This mid-term project requires each team (2–3 students) to take a fully functional web application and deploy it onto a Ubuntu cloud server using two different approaches:

- **Traditional server-based deployment**, and
- **Modern deployment using Docker and Docker Compose** on the same cloud server.

The purpose of this project is to ensure students gain practical experience with DevOps-oriented software delivery, including repository management, Linux automation, cloud deployment, firewall configuration, reverse proxy setup, domain & HTTPS provisioning, containerization, image registry usage, and deployment comparison.

Through this project, students must demonstrate not only that their application “runs,” but that it is deployed correctly, securely, reproducibly, and professionally, following DevOps best practices.

A final comparative analysis between the two deployment methods is required in the technical report and the presentation.

2. Application Requirements

Teams may freely choose any technology stack, programming language, or framework. Please note that the coding section will not be graded. You can use the source code of an existing web project or use AI to create a web project suitable for deployment to the cloud in this midterm assignment. The application must satisfy the following minimum functional requirements:

- A fully working web interface (frontend + backend or monolithic).
- A database component (MySQL, PostgreSQL, MongoDB, etc.).
- A file upload feature that processes uploaded files through backend logic.

Alternatively, you can also use the project provided by the instructor in the **sample-midterm-project.zip** file attached to this document as a starting point. Remember to carefully read the included **README.md** file to understand how to use it. The application must run correctly on local machines before deployment begins.

3. Git/GitHub Collaboration Requirements (Mandatory)

Each team must use GitHub (or GitLab with equivalent features) following professional collaboration practices:

1. All members must appear in the commit history with meaningful contributions.
2. **Branch protection** must be enabled on the main branch, including:
 - Required pull requests before merging
 - At least one reviewer per pull request
3. All merges into the main branch must be done via **Pull Requests** (PRs), never direct commits.
4. PRs must contain clear titles, descriptions, and evidence of review.
5. Technical report must include screenshots or links showing:
 - Branch protection settings
 - PR history and reviews
 - Commit history with all team members through the **Github Insight > Contributors** feature.
 - Repository structure and organization

4. Phase 1 – Git Workflow and Linux Automation

Phase 1 constitutes the foundational stage of the mid-term project and serves to evaluate each team's ability to maintain a professionally structured repository, adhere to contemporary software collaboration practices, and prepare essential automation tools for subsequent deployment phases. The activities in this phase must be completed prior to initiating any form of cloud-based deployment.

4.1 Repository Structure and Documentation Standards

Although individual technology stacks may prescribe their own internal project layouts, every team is expected to present a repository that reflects a clear and disciplined organizational ethos. Regardless of the framework or architectural style employed, certain universal components must be present. A comprehensive **README** file, written in professional technical English, is required to introduce the project's purpose, summarize the technology stack, describe the steps necessary to run the application locally, document the relevant environment variables, and outline—in a high-level manner—the approach that will be taken in later deployment phases. The README should also provide a brief explanation of the automation scripts included in the repository so that other developers may reproduce the setup process with minimal ambiguity.

In addition, each repository must be accompanied by a correctly configured **.gitignore** file. This file should exclude all framework-specific build artifacts, temporary files, dependency folders, generated outputs, log files, and any form of configuration that reveals credentials or other sensitive information. Students may employ the official GitHub templates for their chosen framework as long as the final configuration effectively prevents accidental inclusion of non-essential files in version control.

To support automation, a dedicated folder—typically named **scripts**—must be included. This folder will contain all shell scripts developed as part of Phase 1 and will be reused during the manual deployment process in Phase 2. Although optional, teams are encouraged to maintain a supplementary documentation directory, such as a docs folder, for storing diagrams, screenshots, or other materials that will later appear in the technical report.

4.2 Collaborative Git Workflow and Branch Protection Policies

Phase 1 requires each team to adopt a disciplined collaboration model that aligns with standard professional practices in DevOps-oriented software development. All development work must occur in separate feature branches rather than through direct modification of the main branch. Merging changes into the main branch must be conducted exclusively through pull requests, each accompanied by a descriptive title, a clear summary of the modifications introduced, and at least one formal review from another team member.

The main branch must additionally enforce branch protection policies that prevent direct commits and disallow force-push operations. The pull-request requirement must be active at all times. Contribution patterns should reflect consistent engagement from all members across the duration of the project rather than a concentrated effort immediately before submission. As part of the technical report, teams must present evidence of their adherence to these policies, including screenshots that document branch protection settings, representative pull requests with reviewer approval, and a commit history demonstrating meaningful contributions from all participants.

4.3 Linux Automation Script Requirements

Phase 1 also requires teams to develop a functional automation script, typically implemented as **setup.sh**, capable of preparing a Ubuntu server with the runtime environment necessary to execute the application. The script must install all system dependencies required by the project's technology stack, including appropriate language runtimes and any additional packages essential for compilation or execution. It must also create the directory structures required by the application, such as folders for logs, uploads, or persistent data, and should include concise explanatory comments to clarify the purpose of each major step.

The script must not contain hard-coded credentials, tokens, or sensitive configuration values. While the script is expected to streamline and standardize the preparation of a project environment, Phase 1 does not require idempotent behavior, advanced logging features, or testing on a fresh Ubuntu instance. Instead, the emphasis rests on clarity, correctness, and the practical usefulness of the script in facilitating the deployment tasks that follow in Phase 2.

4.4 Summary of Expected Outcomes

Upon completion of Phase 1, each team must present a repository that is organized in a professional manner, documented clearly and comprehensively, and governed through a modern collaborative workflow. The .gitignore configuration must effectively prevent the inclusion of unnecessary or sensitive files, while the README must convey sufficient information for another developer to understand and execute the project without external guidance. The Git workflow must be demonstrably enforced through pull requests and branch protection mechanisms, with all members contributing substantively to the project. Finally, the Linux automation script must be fully operational and capable of preparing the application environment in a manner consistent with contemporary DevOps practices.

5. Phase 2 – Traditional Deployment on Ubuntu Cloud Server

Phase 2 requires each team to deploy the application directly on a **cloud-based Ubuntu server** without the use of containerization technologies. This phase is intended to develop students' competence in performing conventional server administration tasks, configuring runtime environments, managing security settings, and enabling reliable operational behaviour under real-world conditions. The activities undertaken in Phase 2 must preserve a clear conceptual distinction from Phase 3, in which the same application will be migrated to a container-based deployment approach. For this reason, all software components executed in Phase 2 must run natively on the host operating system.

5.1 Provisioning and Securing the Cloud Server

Each team must provision an Ubuntu server instance from a cloud provider of their choice, such as AWS, Azure, Google Cloud, DigitalOcean, or an equivalent platform (even with providers from Vietnam). Once provisioned, the server must be configured according to the principle of least privilege. Only the necessary ports—typically SSH for administrative access, and ports 80 and 443 for HTTP and HTTPS traffic—may remain open. Any deviation from these default allowances must be justified explicitly in the technical report. SSH access must not be exposed indiscriminately to the public internet; when broader exposure is unavoidable, the decision must be documented along with the associated mitigation strategy.

5.2 Preparing the Runtime Environment

The automation script developed during Phase 1 must be used to prepare the essential runtime environment on the cloud server. Although the script is not required to perform a complete automated deployment, it must reliably install the language runtimes, framework dependencies, and operating-system packages that the application requires. Additional preparation steps—such as installing build tools, configuring directory structures, or generating placeholder configuration files—may be performed either through the script or through documented manual procedures. The primary objective at this stage is to ensure that the cloud server is able to execute the application in a stable and predictable manner.

5.3 Database Configuration and Connectivity

Students may select any database provisioning method they deem appropriate for their application, including the use of a managed cloud database service, a database hosted on another provider, or an instance installed directly on the same Ubuntu server. The project template supplied by the instructor already includes database connectivity logic, and students using this template must simply provide valid credentials via an environment file (.env) in order to establish a successful connection. Students who choose to employ their own codebase may configure the database in a manner consistent with their framework.

Although database setup is not graded as a standalone feature, the successful operation of the deployed system depends on establishing a functioning database connection. In cases where the supplied template falls back to an in-memory storage mode due to a failed connection, this behaviour will be visible and will affect the assessment of system completeness and correctness. Students using their own codebases bear full responsibility for ensuring that the application behaves gracefully under misconfiguration or database unavailability.

5.4 Deploying and Operating the Application

Once the runtime environment and database configuration are in place, students must deploy and operate the application directly on the host system. The application must be executed through a mechanism that ensures continued operation in the event of a server reboot. Because frameworks differ significantly in their deployment expectations, the choice of process manager or service supervisor is left to the teams. Solutions such as **PM2**, a **systemd** service unit, or any equivalent supervisory mechanism are all acceptable, provided that they ensure automatic restart and sustained availability of the application. The technical report must describe the chosen method, explain its rationale, and demonstrate that the deployed system fulfills the persistence requirement.

5.5 Reverse Proxy, Domain Configuration, and HTTPS Enablement

A reverse proxy must be configured at the host level—typically using **Nginx** or Apache—to route incoming HTTP and HTTPS traffic to the application. This proxy configuration from Phase 2 will be preserved in Phase 3; only the upstream target will be modified when the application transitions to containerized execution. Each team must acquire **a domain name** from any reputable registrar. Affordable options (e.g. hostinger.vn) are acceptable and encouraged. The domain must be configured to point to the cloud server’s public IP address, and the site must be secured with a valid TLS certificate issued by Let’s Encrypt or an equivalent certificate authority. Automated certificate renewal is not required for this mid-term project, although students may implement it voluntarily. **Certbot** and **Caddy** are two excellent tools for automatically registering HTTPS for your website.

5.6 Expected Operational State

Upon completing Phase 2, the application must be fully functional over HTTPS, including all features related to database operations and file uploads. The system should demonstrate operational stability, respond predictably after server restarts, and exhibit behaviour consistent with the application’s local execution environment. The technical report must provide evidence of the successful deployment, including screenshots of DNS configuration, proxy behaviour, HTTPS status, and the running application.

6. Phase 3 – Docker Installation & Deployment with Docker Compose (Mandatory)

Phase 3 requires students to migrate the application from its traditional host-based deployment model to a fully containerized architecture. In contrast to Phase 2, where services executed directly on the Ubuntu host, this phase mandates that **every component of the system must run within Docker containers**, orchestrated collectively through Docker Compose. This requirement includes both the web application and the database system. For students using the instructor-provided sample project, which integrates backend and frontend logic into a single Node.js Express application, the project is to be treated as a unified “web service” container. For the database, all teams must deploy a **containerized MongoDB instance**, even if they previously relied on an external or cloud-managed database service. Students implementing their own independent projects must containerize each component contained in their architecture, resulting in a Docker Compose configuration that reflects the full service topology of their application.

6.1 Installation and Verification of Docker and Docker Compose

Students must install **Docker** Engine and Docker Compose on the same Ubuntu server used in Phase 2. Both tools must operate correctly, and the technical report must provide evidence demonstrating successful installation, such as version outputs and representative terminal commands. The host-level

reverse proxy configuration from Phase 2 must remain operational and will continue to serve as the public entry point for HTTPS traffic. Under this phase, only the upstream target within the reverse proxy configuration will change, redirecting requests to the containerized web service rather than the host-executed application.

6.2 Containerization of the Web Application

The web application **must** be packaged as a production-ready Docker image. Students must prepare a Dockerfile that builds the application and ensures that it runs in a secure and efficient runtime environment. The resulting image must be built locally and subsequently pushed to a container registry such as Docker Hub. The technical report must include verifiable evidence of the image-building and pushing process. During deployment, Docker Compose must not build the web image directly from the server's local source code; instead, it must pull the previously published image from the registry. This approach reinforces real-world DevOps workflows by separating build and deployment stages.

Students are responsible for updating environment variables and configuration files so that the containerized application interacts correctly with the database container, persistent storage, and reverse proxy routing defined in the compose configuration.

6.3 Containerization of the Database

Phase 3 requires all teams to deploy the database component entirely within Docker. External databases, cloud-managed database services, or host-installed database servers used in Phase 2 must not be reused. Instead, the database must be instantiated as a dedicated service within Docker Compose, typically by referencing a trusted and officially maintained upstream image (e.g., mongo:6.0). This ensures that the entire application stack is self-contained, reproducible, and portable across environments. Students must configure the database service appropriately, including defining environment variables, port mappings (if needed for debugging), and persistent volumes to preserve data across container restarts.

6.4 Persistent Storage and File Upload Handling

If the application supports file uploads, the deployment must ensure that uploaded files persist across restarts of the web container. This is to be achieved through Docker volumes or bind mounts, depending on the architecture. The compose configuration must define persistent volumes for both uploaded data

and the database's internal storage directory. The correctness of this configuration must be demonstrated in the video demo and documented clearly in the technical report.

6.5 Docker Compose Architecture and Service Orchestration

The Docker Compose file must define at least two services: the web application and the database. Additional services may be included if required by the student's custom project. Each service must be configured with appropriate environment variables, networks, and volumes. Docker Compose's automatic networking should be used to ensure internal communication among services, and the report must describe how service names and internal ports are leveraged to achieve stable inter-service connectivity.

The reverse proxy deployed in Phase 2 must continue to operate on the host machine; however, its upstream target must be updated to route traffic into the containerized web service. Students must confirm that HTTPS access remains fully functional and that the transition from host-based execution to container-based execution does not alter the system's externally observable behaviour.

6.6 Service Reliability, Restart Policies, and Server Reboot Behaviour

All services defined in the compose configuration must be configured with appropriate restart policies to ensure continuous and reliable operation. In particular, services must automatically restart when containers stop unexpectedly and after the Docker daemon itself restarts. Furthermore, Docker must be configured to start automatically when the Ubuntu server reboots, thereby guaranteeing that the complete application stack resumes operation without manual intervention. This operational continuity must be demonstrated during the video presentation and discussed in the technical report.

6.7 Expected System State After Migration

Upon completing Phase 3, the entire application must operate exclusively through Docker containers orchestrated by Docker Compose. The web application must be reachable via the domain configured in Phase 2, secured by HTTPS, and fully functional with respect to database operations and file uploads. No part of the application should rely on host-level execution mechanisms such as PM2 or systemd. Students must gather and present evidence confirming correct functionality, including screenshots of running containers, volume bindings, service logs, docker ps outputs, and successful interaction with the system through the public domain.

7. Technical Report Requirements (Strict Requirements)

The Technical Report constitutes an official academic deliverable and must adhere strictly to the structure, formatting conventions, and presentation standards prescribed in the lecturer's project or thesis report template. It must be written in polished, formal English, using complete sentences and well-developed paragraphs; fragmentary notes, bullet-point drafts, or informal prose are not acceptable. The report must demonstrate not only technical competence but also clarity of reasoning, coherence of explanation, and professionalism in documentation.

The report must provide a comprehensive account of the entire mid-term project, structured explicitly around the three implementation phases. Each phase must be presented as a dedicated section or chapter containing: (1) the phase objectives; (2) the methodology adopted; (3) the technical artefacts produced; (4) the resulting system behaviour; and (5) the supporting evidence validating the correctness of the implementation.

For Phase 1, the report must describe the repository's structure, explain the Git workflow employed, justify the branch protection configuration, and present representative pull requests demonstrating collaborative development practices. It must also describe the purpose and behaviour of the automation script and explain how it assists in preparing the runtime environment.

For Phase 2, the report must document the complete deployment procedure for the traditional host-based setup, including the provisioning and securing of the cloud server, the preparation of system runtimes, the database configuration, the deployment and operation of the application, the reverse proxy configuration, and the acquisition and installation of the HTTPS certificate. All configuration files, commands, DNS settings, system behaviours, and verifiable evidence must be clearly explained and contextualized.

For Phase 3, the report must provide a detailed exposition of the containerization strategy. This includes the construction of the Dockerfile, the image build-and-push workflow, the definition of the docker-compose architecture, the containerization of both the web application and the database, the design of the network and volume configuration, and the behaviour of the system under restart policies and server reboot conditions. Evidence must include screenshots of Docker Hub repositories, docker build and docker push operations, container execution states, persistent volume behaviour, and successful HTTPS operation through the reverse proxy.

The report must also contain a dedicated comparative analysis between the Phase 2 and Phase 3 deployments, grounded in empirical observation rather than theoretical abstraction. This comparison should examine maintainability, reproducibility, operational resilience, deployment complexity, security

considerations, and cost implications. The analysis must clearly articulate the practical differences experienced during implementation and not merely restate general DevOps principles.

The division of responsibilities among team members must be described explicitly, accompanied by commit history and pull-request evidence substantiating the stated contributions. Finally, the report must include system architecture diagrams for both deployment models, network diagrams where relevant, and any visual aids necessary for understanding the project's structure and behaviour. The document must be grammatically sound, well-formatted, and professionally presented.

8. Deliverables

All deliverables must be submitted in a structured and professionally organized format. To ensure clarity, reproducibility, and alignment with the three-phase design of the mid-term project, each team must submit a GitHub or GitLab repository that contains, at minimum, three top-level directories named **phase1**, **phase2**, and **phase3**. Each directory must contain all artefacts, configurations, scripts, and evidence relevant to its corresponding phase.

The **phase1** directory must include: (1) the repository structure as finalized after applying professional conventions; (2) the automation script(s); (3) the completed README and .gitignore; (4) all configuration files created during this phase; and (5) screenshots demonstrating branch protection settings, representative pull requests, commit histories, and any additional artefacts evidencing compliance with the Git workflow requirements.

The **phase2** directory must contain all artefacts related to the traditional deployment on the cloud server, such as: (1) configuration files for the reverse proxy; (2) environment variable templates; (3) DNS configuration screenshots; (4) evidence of HTTPS certificate installation; (5) screenshots demonstrating the operational state of the system, including database connectivity and file uploads; and (6) documentation of the process manager configuration ensuring persistence across server reboots.

The **phase3** directory must contain: (1) the finalized Dockerfile(s); (2) the docker-compose project directory, including volumes, network definitions, .env files, and production configuration; (3) screenshots of Docker Hub repositories showing the pushed web application image; (4) evidence of docker build, docker push, and docker pull operations; (5) docker ps outputs demonstrating the correct execution of all services; (6) evidence of persistent volume behaviour; (7) verification that reverse

proxy routing continues to function through HTTPS; and (8) evidence that the containerized system restarts correctly after container restarts, Docker daemon restarts, and full server reboots.

In addition to the repository, teams must submit the following:

1. The official GitHub/GitLab repository link, containing the structured directories and all required artefacts.
2. The Technical Report, written in the lecturer's official academic template.
3. A presentation slide deck that includes architectural diagrams, implementation summaries for each phase, and a formally presented comparative analysis between the two deployment models.
4. A live demonstration during class, showing the Phase 3 containerized system operating over the HTTPS-secured domain.
5. A video demonstration that presents: (a) the running system; (b) Phase 2 and Phase 3 deployment steps; (c) Docker image build-and-push workflow; (d) Docker Compose deployment; (e) evidence of persistence and restart behaviour; and (f) verification of correct HTTPS routing.
6. All required evidence of Git workflow, including PR history, branch protection settings, commit logs, and any additional documentation that demonstrates compliance with the collaborative development requirements.

All deliverables must be complete, accurate, and consistent with the technical and academic expectations outlined throughout the project specification.

9. Notes

- You may choose to rent an Ubuntu server from any local or international provider, with any configuration, as long as it is sufficient to complete the midterm lab requirements.
- Since maintaining an Ubuntu server for offline instructor verification may consume unnecessary resources, groups are allowed to terminate their servers after completing the presentation and submitting the technical report. However, all implemented features must be clearly listed and supported by convincing evidence in the demo video and the technical report.
- As of 12/12/2025, websites such as <https://www.hostinger.com/vn/ten-mien> offer various domain names (e.g., .online, .shop, .store) starting from around USD 1, while still providing full functionality similar to more common domains. Please note that you should disable auto-renew after purchase, as the discounted price applies only to the first year.

Phase / Task	Points	Performance Level	Description
1. Phase 1 – Git Workflow, Repository Quality & Automation	2.0 points	25%	Poor repository organization; insufficient README; incorrect or missing branch protection; PR workflow incomplete or absent; automation script incorrect or barely functional.
		50–75%	Acceptable repository structure; README present but not comprehensive; PR workflow exists but inconsistent; automation script works but lacks clarity or completeness.
		100%	Professionally structured repository; complete and well-written README; fully enforced PR workflow with branch protection; clear contributions from all team members; automation script correct, purposeful, and supportive of Phase 2.
2. Phase 2 – Traditional Deployment, Security & Runtime Stability	4.0 points	25%	Deployment unstable or incomplete; HTTPS or reverse proxy not functioning; application does not restart after reboot; database connection fails (fallback observed).
		50–75%	Deployment mostly functional; HTTPS and proxy configured but with minor issues; restart mechanism partially working; some features behave inconsistently.
		100%	Server correctly provisioned with proper firewall rules; domain, DNS, and HTTPS fully functional; reverse proxy correctly configured; application stable and automatically restarts after reboot; database connection and file uploads work correctly.
3. Phase 3 – Containerization, Dockerfiles, Registry, Volumes & Compose	2.0 points	25%	Incomplete containerization (missing web or DB container); incorrect compose structure; no image pushed to registry; HTTPS fails after migration.
		50–75%	Containerization functional but lacks best practices; compose runs but volume or network configuration is incomplete; registry workflow only partially demonstrated.
		100%	Full containerization of both web and database; web image built and pushed to Docker Hub; compose pulls images correctly; database uses a proper official image; compose includes networks, volumes (uploads + DB), and environment variables; reverse proxy routing works with the container system; system stable after container restart, Docker restart, and full server reboot.
5. Technical Report – Academic Quality, Architecture, Comparative Analysis	0.5 points	25%	Report poorly written; lacks academic structure; missing key sections; weak or absent comparative analysis.
		50–75%	Report acceptable but not fully professional; some diagrams or evidence missing; comparative analysis shallow or partially developed.
		100%	Fully professional academic report following the official template; clearly structured by Phase 1–2–3; contains complete evidence, appendices, and well-designed architecture diagrams; comparative analysis based on real observed behaviour; writing polished and coherent.
6. Video Demonstration	0.5 points	25%	Video incomplete; missing core deployment steps; HTTPS or containerization not demonstrated.
		50–75%	Video mostly correct but missing one or more required demonstrations (e.g., restart behaviour).
		100%	Clear, complete demonstration of the full system: Phase 2 and Phase 3 deployment steps, Docker build/push/pull, docker compose workflow, persistence, restart behaviour (container restart + Docker restart + server reboot), and HTTPS routing.
7. Presentation (Live, In-Class)	1.0 points	25%	Weak presentation; unclear explanations; demo failure; insufficient understanding of system architecture.
		50–75%	Clear presentation but lacks coherence or depth; some architectural details or explanations missing.
		100%	Professional, confident, and well-structured presentation; strong explanation of both Phase 2 and Phase 3 architectures; smooth demo; accurate responses to questions; clear comparison of deployment models.